Webpack 原理解析 - 模块编译阶段

模块编译阶段，`webpack` 的核心流程主要做了以下几件事：

1. 根据入口文件路径分析入口文件，匹配对应的 `loader` 对入口文件进行处理。
2. 将 `loader` 处理完成的入口文件使用 `webpack` 进行编译。
3. 分析入口文件依赖，重复上边两个步骤编译对应依赖。
4. 如果嵌套文件存在依赖文件，递归调用依赖模块进行编译。
5. 递归编译完成后，组装成一个个包含多个模块的 `chunk` 。

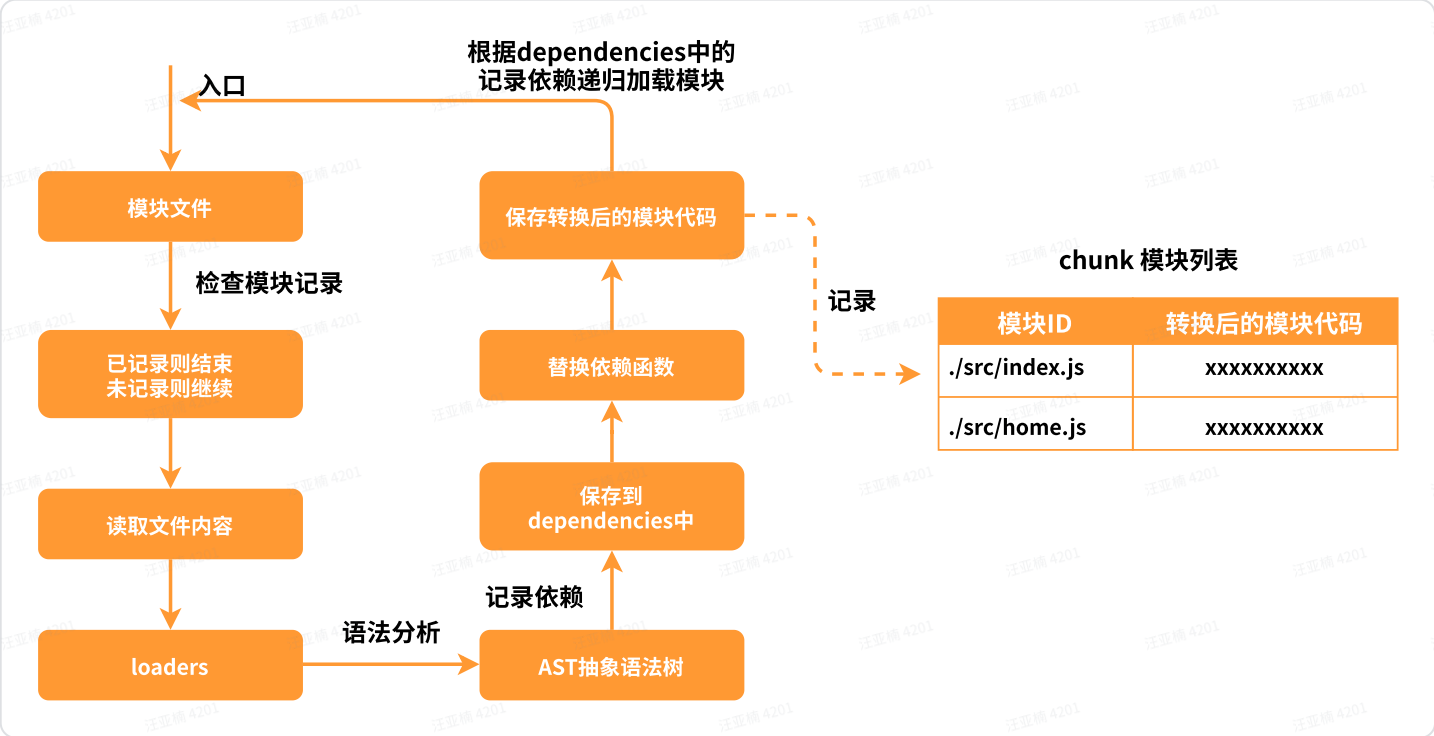
> 💡 `webpack` 在开始构建前，会创建一个 `Compilation` 实例

Compilation

`Compilation` 类实例，提供了 webpack 大部分生命周期Hook API供自定义处理时做拓展使用。一个 `Compilation` 对象记录了一次构建到生成资源过程中的信息，它储存了当前的模块资源、编译生成的资源、变化的文件、以及被跟踪依赖的状态信息。

简单来说，`Compilation` 的职责就是对所有 require 进行编译，编译构建 module 和 chunk，并调用插件构建过程，同时把本次构建编译的内容全存到内存里。`Compilation` 编译可以多次执行，如在watch模式下启动 webpack，每次监测到源文件发生变化，都会重新实例化一个 `Compilation` 对象，从而生成一组新的编译资源对象。这个对象可以访问所有的模块和它们的依赖。

示例流程图：



这个过程中的数据流 `module => ast => dependences => module` ，先转 AST 再从 AST 找依赖。这就要求 `loaders` 处理完的最后结果必须是可以被 acorn 处理的标准 JavaScript 语法，比如说对于图片，需要从图像二进制转换成类似于 `export default ""` 这类 base64 格式或者 `export default "http://xxx"` 这类 url 格式。

`Compilation` 按这个流程递归处理，逐步解析出每个模块的内容以及 `module` 依赖关系，后续就可以根据这些内容打包输出。

以上过程中， `loader` 的处理依赖于 `loader-runner` 库，ast语法解析依赖于 `acorn` 库。

实现自定义模块编译方法（3-1）

完善 `Compiler类` 新增compile方法

```
// core/compiler.js
const Compileation = require('./compilation.js')

// Compiler类进行核心编译实现
class Compiler {
  /**
    * ********************
```

```
     */

    /**
     * run方法启动编译
     * @param {*} callback 运行结束后触发的回调
     */
    run(callback) {
      // 调用run方式 触发开始编译的plugin钩子
      this.hooks.run.call()

      // 开始编译
      this.compile()
    }

    /**
     * ********************
     */

    /**
     * 模块编译方法
     */
    compile() {
      // 编译完成回调
      const onCompiled = (err, compilation) => {}

      const compilation = new Compileation(this)
      compilation.seal(onCompiled)
    }
  }

  module.exports = Compiler
```

新增 `Compilation类`

```
// core/compilation.js
const fs = require('fs')

class Compilation {
  constructor(compiler) {
    // 保存全局编译器
    this.compiler = compiler
    // 保存所有入口模块对象
    this.entries = new Set()
    // 保存所有依赖模块对象
```

```
    this.modules = new Set()
    // 所有的代码块对象
    this.chunks = new Set()
  }

  /**
   * 编译并保存入口文件对象
   */
  buildEntryModule(entry) {
    Object.keys(entry).forEach((entryName) => {
      const entryPath = entry[entryName]
      const entryObj = this.buildModule(entryName, entryPath)
      this.entries.add(entryObj)
    })
  }

  /**
   * 编译单个模块对象
   */
  buildModule(moduleName, modulePath) {
    // 1、通过fs模块读取原始文件内容
    const originSourceCode = fs.readFileSync(modulePath, 'utf8')
    // moduleCode 为修改后的模块代码，后续步骤中会逐渐修改
    this.moduleCode = this.originSourceCode = originSourceCode

    return {}
  }

  /**
   * 处理module到chunk的转化
   * @param {*} callback 转化完成后的回调方法
   */
  seal(callback) {
    this.buildEntryModule(this.compiler.entry)
  }
}

module.exports = Compilation
```

- `buildEntryModule` 方法作为入口模块编译方法。循环入口对象，得到每一个入口对象的名称和路径。
- `buildModule` 方法编译单个模块文件，最终返回一个对象。这个对象就是我们编译转化后的最终内容。

实现自定义处理loader方法（3-2）

```
// webpack.config.js
// 配置文件传入自定义loader方法
module: {
    rules: [
      {
        test: /\.js/,
        use: [
          path.resolve(__dirname, './loaders/my-loader2.js'),
          path.resolve(__dirname, './loaders/my-loader.js')
        ]
      }
    ]
  }
```

`loader` **本质上就是一个函数，接受我们的源代码作为入参同时返回处理后的结果。**

```
// loaders/my-loader.js
function myLoader(sourceCode) {
  console.log('chaser: join my-loader code')
  return `
    ${sourceCode}
    const loader = 'loader join code'
  `
}
module.exports = myLoader

// loader/my-loader2.js
function myLoader2(sourceCode) {
  console.log('chaser: join my-loader2 code')
  return `
    ${sourceCode}
    const loader2 = 'loader2 join code'
  `
}
module.exports = myLoader2
```

```
// core/compilation.js
const fs = require('fs')
```

```

class Compilation {
  /**
   * 编译单个模块对象
   */
  buildModule(moduleName, modulePath) {
    // 1、通过fs模块读取原始文件内容
    const originSourceCode = fs.readFileSync(modulePath, 'utf8')
    // moduleCode 为修改后的模块代码，后续步骤中会逐渐修改
    this.moduleCode = this.originSourceCode = originSourceCode

    // 2、调用loader处理匹配的文件
    this.handleLoader(modulePath)
    return {}
  }

  /**
   * 匹配loader处理
   */
  handleLoader(modulePath) {
    const rules = this.compiler.options.module.rules
    rules.forEach(({ test, use }) => {
      // 1、获取所有匹配的loader规则
      const matchLoaders = []
      if(test.test(modulePath)) {
        // 这里仅处理字符串或字符串数组格式的use
        if (typeof use === 'string') {
          matchLoaders.push(use)
        }else {
          matchLoaders.push(...use)
        }
      }
      // 2、传入源代码倒序执行loader
      for(let i = matchLoaders.length - 1; i >= 0; i--) {
        // 引入loader方法 目前仅支持传入绝对路径
        const loaderFn = require(matchLoaders[i])
        // 通过loader同步处理每一次编译的moduleCode
        this.moduleCode = loaderFn(this.moduleCode)
      }
    })
  }
}

module.exports = Compilation
```

这里我们通过 `handleLoader` 函数，对于传入的文件路径匹配到对应后缀的 `loader` 后，依次倒序执行loader处理我们的代码 `this.moduleCode` 并且同步更新每次的 `this.moduleCode` 。

实现模块语法分析方法（3-3）

@babel/parser 将源代码转化成抽象语法树（AST）

@babel/traverse （遍历AST）模块维护了整棵树的状态，并且负责替换、移除和添加节点

@babel/generator 读取AST并将其转换为代码和源码映射

@babel/types 包含手动构建 AST 和检查 AST 节点类型的方法

```
// core/compilation.js
const path = require('path')
const parser = require('@babel/parser')
const traverse = require('@babel/traverse').default
const generator = require('@babel/generator').default
const t = require('@babel/types')

class Compilation {
  /**
   * 编译单个模块对象
   */
  buildModule(moduleName, modulePath) {
    // 1、通过fs模块读取原始文件内容
    const originSourceCode = fs.readFileSync(modulePath, 'utf8')
    // moduleCode 为修改后的模块代码，后续步骤中会逐渐修改
    this.moduleCode = this.originSourceCode = originSourceCode

    // 2、调用loader处理匹配的文件
    this.handleLoader(modulePath)

    // 3、模块语法分析 获得最终的module对象
    const module = this.handleWebpackCompiler(moduleName, modulePath)
    return module
  }

  /**
   * 处理模块，收集依赖，通过语法分析替换require方法
   * @param {*} moduleName 模块名称
   * @param {*} modulePath 模块路径
   */
  handleWebpackCompiler(moduleName, modulePath) {
    // 1、将当前模块相对于项目启动根目录计算出相对路径 作为模块ID
    const moduleId = './' + path.posix.relative(process.cwd(), modulePath)
```

```
    // 创建模块对象
    const module = {
      id: moduleId,
      dependencies: new Set(), // 该模块所依赖模块绝对路径地址
      name: [moduleName], // 该模块所属的入口文件
    }
    // 将源代码转化成抽象语法树
    const ast = parser.parse(this.moduleCode, {
      sourceType: 'module',
    })
    // 深度优先遍历抽象语法树
    traverse(ast, {
      // 调用表达式触发
      CallExpression: (nodePath) => {
        const node = nodePath.node
        // 判断是否调用require语句
        if (node.callee.name === 'require') {
          // 获取依赖模块引入相对路径
          const requirePath = node.arguments[0].value
          // 转化模块绝对路径
          const moduleDirName = path.posix.dirname(modulePath)
          // TODO: webpack中通常会在转化为绝对路径前处理无后缀的文件
          const absolutePath = path.posix.join(moduleDirName, requirePath)
          // 生成moduleId - 相对于启动路径的模块ID
          const moduleId = './' + path.posix.relative(process.cwd(), absolutePath)
          // 记录依赖
          module.dependencies.add(moduleId)
          // 修改源代码中的require变成__webpack_require__语句
          node.callee = t.identifier('__webpack_require__')
          // 修改源代码中的require语句引入模块 相对于根路径来处理
          node.arguments = [t.stringLiteral(moduleId)]
        }
      }
    })
    // 遍历结束根据AST生成新的代码
    const { code } = generator(ast)
    // 为当前模块挂载解析后的代码
    module._source = code
    // 返回整个模块对象
    return module
  }

  /**
   * 处理module到chunk的转化
   * @param {*} callback 转化完成后的回调方法
   */
```

```
  seal(callback) {
    this.buildEntryModule(this.compiler.entry)
  }
}

module.exports = Compilation
```

对于每个模块的编译，我们都会返回一个 `module` 对象，这个对象是重中之重。

- `id` 属性，表示当前模块针对于根路径的相对路径。
- `dependencies` 属性，它是一个 `Set` 内部保存了该模块依赖的所有模块的模块ID。
- `name` 属性，它表示该模块属于哪个入口文件。
- `_source` 属性，它存放模块自身经过 `babel` 编译后的字符串代码。

实现递归处理依赖模块方法（3-4）

截止目前，我们已经完成了入口文件的全部编译过程，接下来，就是递归编译入口文件的依赖模块即可，我们把依赖模块当做一个入口文件处理，调用 `buildModule` 方法进行编译后加入 `modules` 模块列表即可。

完善 `handleWebpackCompiler()` 方法

```
// core/compilation.js

/**
* 处理模块，收集依赖，通过语法分析替换require方法
* @param {*} moduleName 模块名称
* @param {*} modulePath 模块路径
*/
handleWebpackCompiler(moduleName, modulePath) {
// 1、将当前模块相对于项目启动根目录计算出相对路径 作为模块ID
const moduleId = './' + path.posix.relative(process.cwd(), modulePath)
// 创建模块对象
const module = {
  id: moduleId,
  dependencies: new Set(), // 该模块所依赖模块绝对路径地址
  name: [moduleName], // 该模块所属的入口文件
}
// 将源代码转化成抽象语法树
const ast = parser.parse(this.moduleCode, {
  sourceType: 'module',
})
// 深度优先遍历抽象语法树
```

```
traverse(ast, {
  // 调用表达式触发
  CallExpression: (nodePath) => {
    const node = nodePath.node
    // 判断是否调用require语句
    if (node.callee.name === 'require') {
      // 获取依赖模块引入相对路径
      const requirePath = node.arguments[0].value
      // 转化模块绝对路径
      const moduleDirName = path.posix.dirname(modulePath)
      // TODO: webpack中通常会在转化为绝对路径前处理无后缀的文件
      const absolutePath = path.posix.join(moduleDirName, requirePath)
      // 生成moduleId - 相对于启动路径的模块ID
      const moduleId = './' + path.posix.relative(process.cwd(), absolutePath)
      // 记录依赖
      module.dependencies.add(moduleId)
      // 修改源代码中的require变成__webpack_require__语句
      node.callee = t.identifier('__webpack_require__')
      // 修改源代码中的require语句引入模块 相对于根路径来处理
      node.arguments = [t.stringLiteral(moduleId)]
    }
  }
})
// 遍历结束根据AST生成新的代码
const { code } = generator(ast)
// 为当前模块挂载解析后的代码
module._source = code

// 递归深度遍历依赖，将编译后的依赖模块添加到modules中
module.dependencies.forEach((dependency) => {
  const depModule = this.buildModule(moduleName, dependency)
  this.modules.add(depModule)
})

// 返回整个模块对象
return module
}
```

在该方法原来的基础上添加了一段代码

```
// 递归深度遍历依赖，将编译后的依赖模块添加到modules中
module.dependencies.forEach((dependency) => {
  const depModule = this.buildModule(moduleName, dependency)
  this.modules.add(depModule)
})
```

> 💡 至此我们的依赖模块也全部递归编译完成了。但是这里会产生一个问题，如果一个模块多次引用，会导致多次编译，我们需要在刚开始编译的时候检查下该模块是否已记录在模块列表中。

完善 `buildModule()` 方法

```
// core/compilation.js

buildModule(moduleName, modulePath) {
    // 检查模块是否已记录
    let record = false
    this.modules.forEach((module) => {
      if (module.id === modulePath) {
        record = true
        // 更新这个模块依赖的入口
        module.name.push(moduleName)
      }
    })
    // 已记录模块，直接返回
    if (record) return

    // 1、通过fs模块读取原始文件内容
    const originSourceCode = fs.readFileSync(modulePath, 'utf8')
    // moduleCode 为修改后的模块代码，后续步骤中会逐渐修改
    this.moduleCode = this.originSourceCode = originSourceCode

    // 2、调用loader处理匹配的文件
    this.handleLoader(modulePath)

    // 3、模块语法分析 获得最终的module对象
    const module = this.handleWebpackCompiler(moduleName, modulePath)
    return module
}
```

如果记录不存在则添加进入依赖中进行编译，如果该模块已经存在过了就证明这个模块已经被编译过了。所以此时我们不需要将它再次进行编译，我们仅仅需要更新这个模块所属的chunk，为它的 `name` 属性添加当前所属的 `chunk` 名称。

END